RAJALAKSHMI ENGINEERING COLLEGE

Chandrakanth D 251001039-2025-IT

C2

Started on	Wednesday, 7 October 2026, 10:14 PM
State	Finished
Completed on	Wednesday, 7 October 2026, 10:15 PM
Time taken	33 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
#include <stdlib.h>

// Helper comparison function for qsort
int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

void solve() {
    int n, m;

    // Read size of first array and its elements
    if (scanf("%d", &n) != 1) return;
    int *A = (int *)malloc(n * sizeof(int));
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }

    // Read size of second array and its elements
    if (scanf("%d", &m) != 1) {
        free(A);
        return;
    }
    int *B = (int *)malloc(m * sizeof(int));
    for (int i = 0; i < m; i++) {
        scanf("%d", &B[i]);
    }

    // Sort both arrays to find common elements efficiently
    qsort(A, n, sizeof(int), compare);
    qsort(B, m, sizeof(int), compare);

    int i = 0, j = 0;
    int first_print = 1;

    // Use two pointers to find the intersection
    while (i < n && j < m) {
        if (A[i] < B[j]) {
            i++;
        } else if (A[i] > B[j]) {
            j++;
        } else {
            // Found a common element
            if (!first_print) {
                printf(" ");
            }
            printf("%d", A[i]);
            first_print = 0;

            int current = A[i];
            // Skip duplicates to handle unique intersections correctly
            while (i < n && A[i] == current) i++;
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course